

CENTRO UNIVERSITÁRIO DE BALSAS - UNIBALSAS
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

GÉRON RESPLANDES DE SÁ SOUSA

**AUTOMAÇÃO DE PIPELINES CI/CD COM GITHUB ACTIONS E SEUS
IMPACTOS NA ESCALABILIDADE OPERACIONAL DE APLICAÇÕES**

Relatório parcial de pesquisa

Balsas - MA
2026

GÉRSON RESPLANDES DE SÁ SOUSA

**AUTOMAÇÃO DE PIPELINES CI/CD COM GITHUB ACTIONS E SEUS
IMPACTOS NA ESCALABILIDADE OPERACIONAL DE APLICAÇÕES**

Relatório parcial de pesquisa apresentado à unidade curricular Análise e Engenharia de Sistemas, do curso de Análise e Desenvolvimento de Sistemas do Centro Universitário de Balsas - UNIBALSAS, como desenvolvimento do plano de pesquisa do período 2026.2.

Área: Engenharia de Software.

Orientador: Marcos David Souza Ramos.

Acadêmico: Gérson Resplandes de Sá Sousa

RA: 26.1.13332

Balsas - MA

2026

RESUMO

A automação da integração e da entrega de software constitui uma estratégia para organizar verificações e reduzir a variabilidade de procedimentos. Esta pesquisa analisa como pipelines CI/CD com GitHub Actions podem contribuir para a padronização, a rastreabilidade, a confiabilidade e a escalabilidade operacional de aplicações. O estudo é de natureza aplicada, com abordagem predominantemente qualitativa e caráter exploratório e descritivo. Nesta etapa, foram realizadas revisão narrativa da literatura, análise documental de recursos da plataforma e execução de um protótipo local em Python, com seis testes unitários, empacotamento e implantação simulada em diretório de teste. O ensaio compreendeu três execuções da versão inicial, uma execução com defeito deliberadamente introduzido e uma execução após correção, além de uma referência por comandos separados. As três repetições iniciais produziram artefatos com o mesmo resumo criptográfico. O defeito provocou três falhas nos seis testes e impediu build, implantação e verificação posterior; a correção restabeleceu a conclusão do fluxo. A referência por comandos separados também concluiu as etapas, demonstrando que a vantagem observada reside na coordenação e no bloqueio programado, e não em uma incapacidade inerente ao procedimento manual. Os achados locais sustentam a viabilidade técnica de uma sequência repetível com evidências por versão, mas não comprovam redução de erros humanos nem aumento de capacidade em produção. Foram executados também três cenários no GitHub Actions: versão válida, defeito deliberado e correção, com resultados concordantes com o ensaio local. O repositório público e os links das execuções permitem conferir os registros. A observação de filas e o aumento da complexidade permanecem como etapas seguintes.

Palavras-chave: engenharia de software; integração contínua; entrega contínua; GitHub Actions; escalabilidade operacional.

ABSTRACT

Automated software integration and delivery can organize verification activities and reduce procedural variation. This study examines how CI/CD pipelines using GitHub Actions may contribute to standardization, traceability, reliability, and operational scalability. It is an applied, predominantly qualitative, exploratory and descriptive investigation. This stage combines a narrative literature review, documentary analysis of platform features, and a local Python prototype with six unit tests, packaging, and simulated deployment to a test directory. The exploratory trial comprised three executions of the initial version, one execution containing a deliberately introduced defect, and one execution after correction, together with a separate-command reference procedure. The three initial repetitions generated artifacts with identical cryptographic digests. The defect caused three of six tests to fail and prevented packaging, deployment, and the subsequent smoke check. After correction, all stages completed successfully. The separate-command procedure also completed successfully; therefore, the observed benefit concerns coordination and programmed blocking rather than an inherent inability of manual procedures. These local observations support the technical feasibility of repeatable execution with version-specific evidence, but do not establish reductions in human error or increased production capacity. Three scenarios were also executed on GitHub Actions: a valid version, a deliberately faulty version, and its correction. Their results were consistent with the local trial. A public repository and run URLs provide verifiable records. Queue observation and increasing workflow complexity remain subsequent research activities.

Keywords: software engineering; continuous integration; continuous delivery; GitHub Actions; operational scalability.

SUMÁRIO

1 INTRODUÇÃO	6
1.1 Problema e hipótese	6
1.2 Objetivos	6
1.3 Justificativa e delimitação da entrega	7
2 REVISÃO DE LITERATURA	8
2.1 Engenharia de Software e práticas contínuas	8
2.2 Integração, entrega e implantação contínuas	8
2.3 GitHub Actions e configuração como código	9
2.4 Escalabilidade operacional e arquitetura	10
2.5 Segurança, evidências e limites da plataforma	10
3 METODOLOGIA	11
3.1 Natureza, abordagem e desenho	11
3.2 Procedimento bibliográfico e documental	11
3.3 Ambiente e protótipo	12
3.4 Procedimentos de comparação e cenários	13
3.5 Critérios de análise e evidências	14
3.6 Ameaças à validade e procedimentos pendentes	14
4 RESULTADOS PRELIMINARES	15
4.1 Evolução em relação ao plano de pesquisa	15
4.2 Achados bibliográficos e documentais	15

4.3 Resultados do ensaio local	16
4.4 Rastreabilidade e repetibilidade	17
4.5 Comparação com o procedimento de referência	17
4.6 Discussão da hipótese e limites dos achados	18
4.7 Execuções reais e provas públicas no GitHub	18
5 CONSIDERAÇÕES PARCIAIS E CONTINUIDADE	20
REFERÊNCIAS	21
APÊNDICE A - WORKFLOW EXECUTADO NO GITHUB ACTIONS	22
APÊNDICE B - ROTEIRO DE REPRODUÇÃO DO ENSAIO	23
APÊNDICE C - REGISTRO SINTÉTICO DAS EVIDÊNCIAS	24
APÊNDICE D - TRANSPARÊNCIA DA ELABORAÇÃO	25
APÊNDICE E - LINKS PARA CONFERÊNCIA DAS PROVAS	26

1 INTRODUÇÃO

A evolução de uma aplicação exige coordenar alterações no código, verificações de qualidade e procedimentos de entrega. Quando essas atividades dependem de sequências informais de comandos, torna-se difícil demonstrar qual versão foi verificada, quais etapas foram concluídas e por que uma entrega foi autorizada. O problema de pesquisa situa-se nessa relação entre organização do processo e confiança nas mudanças, conforme o plano de pesquisa previamente elaborado.¹

A integração contínua pressupõe integração frequente do trabalho ao código compartilhado, acompanhada de build e testes automatizados. Fowler associa essa prática à identificação rápida de problemas de integração. A entrega contínua, por sua vez, mantém o software em condição de ser liberado sob demanda; a implantação contínua acrescenta a liberação automática para produção.^{2,3}

O estudo investiga a automação de pipelines CI/CD com GitHub Actions. Seu recorte é a escalabilidade operacional do desenvolvimento e da entrega: a capacidade de preservar padronização, rastreabilidade, repetibilidade e confiabilidade diante de maior frequência ou complexidade de mudanças. Esse recorte não equivale à capacidade de atender mais usuários simultâneos, que dependeria de ensaios de carga e da infraestrutura de execução.

1.1 Problema e hipótese

O problema que orienta a investigação é: como a automação de pipelines CI/CD com GitHub Actions pode contribuir para melhorar a padronização, a rastreabilidade, a confiabilidade e a escalabilidade operacional no desenvolvimento e na implantação de aplicações?

A hipótese é que a automação tende a reduzir omissões procedimentais, tornar explícita a sequência de verificações e associar evidências às alterações entregues. Seus benefícios dependem da adequação dos testes, do tratamento de falhas, das permissões e da configuração do fluxo. Nesta etapa, a hipótese é examinada parcialmente por análise documental e por um ensaio local; os resultados ainda não permitem confirmar efeitos organizacionais ou a redução de erros humanos.

1.2 Objetivos

O objetivo geral é analisar os impactos da automação de pipelines CI/CD com GitHub Actions na padronização, na rastreabilidade, na confiabilidade e na escalabilidade operacional de aplicações.

Os objetivos específicos são: diferenciar integração, entrega e implantação contínuas; identificar funcionalidades da plataforma; descrever uma sequência de preparação, testes, build e deploy; estabelecer critérios observáveis de comparação; investigar limitações de configuração e segurança; e propor recomendações proporcionais às evidências obtidas. Esses objetivos preservam o escopo do plano e orientam a seleção das fontes e dos cenários locais.

1.3 Justificativa e delimitação da entrega

A relevância acadêmica está em transformar expectativas sobre automação em critérios verificáveis. Uma indicação visual de sucesso, isoladamente, não explica a qualidade do software: é necessário conhecer o que foi testado, a versão utilizada e os mecanismos que impediram a continuidade após uma falha. A relevância prática está em fornecer um procedimento reproduzível para projetos que precisam evoluir sem perder controle sobre suas entregas.

A relação com o ODS 9 ocorre pelo estudo de processos tecnológicos orientados à inovação e à confiabilidade da infraestrutura digital. A Organização das Nações Unidas inclui infraestrutura resiliente, inovação e fortalecimento da pesquisa entre os componentes desse objetivo. A contribuição deste trabalho é localizada no processo de engenharia; não foram medidos indicadores sociais, ambientais ou econômicos do ODS.⁴

A entrega apresenta revisão de literatura, metodologia, achados documentais e resultados de um protótipo executado localmente em 30 de setembro de 2026. O workflow também foi executado no GitHub Actions em três cenários técnicos, apresentados separadamente na seção 4.7. Nenhum desses registros corresponde a implantação produtiva ou observação de uma equipe de desenvolvimento.

2 REVISÃO DE LITERATURA

2.1 Engenharia de Software e práticas contínuas

Neste trabalho, a Engenharia de Software é abordada pela organização do ciclo de verificação e entrega. Código, testes, scripts de empacotamento e evidências precisam formar uma cadeia compreensível: a alteração identifica uma versão; a verificação aplica critérios; o artefato resulta do conteúdo verificado; e a entrega registra o destino. Essa cadeia constitui o modelo analítico adotado para examinar a automação.

A revisão sistemática de Shahin, Babar e Zhu selecionou 69 estudos publicados entre 2004 e junho de 2016. Os autores identificaram 30 abordagens e ferramentas relacionadas às práticas contínuas e destacaram testes, transparência da equipe, projeto do software e infraestrutura como fatores de adoção. A revisão também apontou insuficiência na descrição dos contextos e lacunas relativas à segurança dos pipelines.⁵

A síntese fornece uma base histórica para investigar mecanismos de controle, mas não avalia os recursos atuais do GitHub Actions. Por isso, o presente trabalho combina literatura acadêmica com documentação técnica consultada na data da pesquisa. A documentação especifica funcionalidades disponíveis; os artigos ajudam a interpretar condições e limitações de adoção. Nenhuma dessas fontes, isoladamente, substitui a verificação do protótipo.

2.2 Integração, entrega e implantação contínuas

Fowler destaca integração frequente à linha principal, build automatizado, testes e tratamento rápido de quebras. Dispor de um servidor de CI não garante, por si só, a prática de integração contínua: o comportamento de integração da equipe também importa. Neste estudo, o protótipo demonstra mecanismos técnicos de verificação, sem observar a frequência de integração de desenvolvedores.²

A DORA distingue a disponibilidade de uma versão liberável sob demanda da implantação automática de cada mudança em produção. Assim, uma aprovação humana pode fazer parte da entrega contínua. O elemento distintivo da implantação contínua é a promoção automática das mudanças aprovadas pelas verificações ao ambiente produtivo.³

Quadro 1 - Distinção operacional entre práticas contínuas

Prática	Foco	Limite do protótipo
Integração contínua	Verificar alterações integradas por build e testes.	Execução técnica local; sem observação de integração cotidiana.
Entrega contínua	Manter uma versão apta à liberação sob demanda.	Empacotamento e entrega a diretório simulado; aptidão produtiva não validada.
Implantação contínua	Promover automaticamente alterações verificadas à produção.	Não realizada: não há destino produtivo nem usuários.

Fonte: elaboração própria a partir de Fowler e DORA; referências 2 e 3.

O termo CI/CD será utilizado como designação geral da cadeia de integração e entrega. Quando houver distinção necessária, serão empregadas as expressões completas. A cópia de um pacote para um diretório local é uma simulação de entrega, e não evidência de implantação contínua em produção.

2.3 GitHub Actions e configuração como código

O GitHub Actions organiza automações em workflows definidos em YAML e associados a eventos do repositório. Cada workflow reúne jobs, executados em runners; os jobs contêm steps que chamam comandos ou actions reutilizáveis. A plataforma oferece runners hospedados e permite runners próprios. Essa estrutura conecta a configuração da automação ao repositório de software.⁶

A documentação de sintaxe estabelece arquivos em `.github/workflows` e propriedades para eventos, permissões, ambiente de execução e dependências entre jobs. O modelo permite versionar a sequência de verificação junto ao código. A inferência adotada aqui é que uma alteração do fluxo passa a ser examinável como alteração de configuração, em vez de permanecer apenas na memória do executor.⁷

O workflow deste estudo utiliza eventos `push`, `pull_request` e `workflow_dispatch`. O processamento central chama um script que executa as cinco etapas e registra seus resultados. A opção por um script compartilhado favorece a comparação local e remota; os logs da plataforma agrupam a sequência em um step, enquanto o script preserva arquivos separados por etapa. Essa escolha de instrumentação será mantida explícita na análise.

2.4 Escalabilidade operacional e arquitetura

Shahin e colaboradores investigaram arquitetura para entrega e implantação contínuas com entrevistas de 21 profissionais de 19 organizações e um levantamento com 91 profissionais. O estudo discute unidades de implantação independentes e atributos como testabilidade, monitorabilidade e resiliência. Também mostra que sistemas monolíticos podem adotar práticas contínuas, embora apresentem desafios específicos.⁸

Esses achados indicam que a avaliação precisa observar o contexto da aplicação. Para este estudo, ampliar a escala operacional significa examinar se a sequência preserva seus controles quando crescem as mudanças, os ambientes ou as verificações. Essa definição será operacionalizada por consistência das etapas, identificação da versão, bloqueio de falhas e conservação das evidências. O ensaio atual verifica apenas um módulo pequeno e um único destino simulado.

Aumento de frequência pode gerar filas ou concorrência entre entregas. O controle de concurrency do GitHub Actions limita execuções dentro de um grupo e pode cancelar execuções em andamento. Para o protótipo, cancel-in-progress permite substituir verificações de uma referência por outras mais recentes. Essa configuração não será interpretada como demonstração de escalabilidade; seu efeito precisará ser observado remotamente.⁹

2.5 Segurança, evidências e limites da plataforma

A documentação de segurança recomenda permissões mínimas para GITHUB_TOKEN, cuidados com segredos e revisão das actions utilizadas. Ela também recomenda fixar actions por SHA completo para maior controle de sua versão. No protótipo publicado, contents: read e persist-credentials: false limitam o uso de credenciais; não há segredo de implantação. As actions utilizadas foram fixadas por SHA completo, conferido a partir das versões oficiais.¹⁰

Artefatos de workflow podem conservar saídas de build e testes depois da execução. Isso permite reunir resultados e produtos em um conjunto consultável, desde que se definam nomes e retenção. O workflow preservou evidence/ e dist/, inclusive os registros de falha, em artefatos configurados para retenção de 14 dias. Cópias dos logs e resumos JSON foram também publicadas no repositório para consulta posterior à expiração dos artefatos.¹¹

Limites de execução, concorrência e armazenamento variam conforme o tipo de runner e a conta; a documentação informa que esses limites podem mudar. Por isso, o relatório não fixa custos ou capacidade de execução como resultado. O timeout de dez minutos proposto é um controle do protótipo. A continuidade da pesquisa exigirá registrar o plano da conta, a configuração dos runners e as condições vigentes.¹²

3 METODOLOGIA

3.1 Natureza, abordagem e desenho

Mantém-se a classificação prevista no plano: pesquisa aplicada, predominantemente qualitativa, exploratória e descritiva. O objeto é o processo de integração e entrega. O desenho desta etapa articula revisão narrativa, análise documental, ensaio exploratório local e três cenários hospedados no GitHub Actions. Contagens de testes e execuções descrevem o caso; não constituem inferência estatística nem alteram a predominância qualitativa da análise.

A unidade de observação é uma execução da sequência de preparação, testes, build, deploy simulado e smoke test. A comparação relaciona o procedimento por comandos separados a um script coordenador que interrompe o fluxo quando uma etapa retorna código de saída diferente de zero. O estudo não inclui participantes humanos, entrevistas, dados pessoais de usuários ou serviços produtivos.

3.2 Procedimento bibliográfico e documental

A consulta foi realizada em 30 de setembro de 2026. Partiu-se das referências do plano e foram acessados textos de Fowler, DORA, dois artigos disponíveis no arXiv e páginas oficiais do GitHub Docs. A busca complementar incluiu as expressões "continuous integration delivery deployment systematic review" e "GitHub Actions limits". A consulta às orientações de apresentação acadêmica foi separada do corpus temático.

Foram incluídas fontes com autoria identificável, aderência ao problema e acesso suficiente para conferir os conceitos utilizados. Para os artigos, foram lidos os resumos e trechos de fundamentos, resultados ou discussão disponíveis nos textos integrais. Foram excluídos, como fundamento técnico, conteúdos comerciais sem procedimento verificável e resultados de busca que não foram efetivamente examinados. Os livros previstos no plano não foram utilizados para atribuir afirmações específicas nesta entrega, pois suas edições integrais não foram consultadas.

Trata-se de seleção intencional e narrativa, sem pretensão de esgotar a produção científica. Não foram realizadas buscas completas em todas as bases bibliográficas, dupla seleção independente ou avaliação formal de risco de viés. As 69 publicações referidas na revisão de 2017 pertencem à amostra dos autores, não à amostra do presente estudo. O corpus temático reúne duas publicações acadêmicas, dois textos conceituais e sete páginas técnicas; o documento da ONU e o plano oferecem contexto.

Quadro 2 - Fontes centrais e função analítica

Grupo	Fontes consultadas	Uso na pesquisa
Literatura acadêmica	Shahin, Babar e Zhu (2017); Shahin et al. (2018).	Condições de adoção e relação com arquitetura.
Fundamentação conceitual	Fowler (2024); DORA (s. d.).	Diferenciação entre integração, entrega e implantação.
Documentação técnica	GitHub Docs: conceitos, sintaxe, concorrência, segurança, artefatos e limites; Python unittest.	Especificação do workflow, instrumentação e execução dos testes.
Contextualização	Plano de pesquisa (2026); ONU, ODS 9.	Escopo, objetivos e relação com inovação.

Fonte: elaboração própria (2026).

3.3 Ambiente e protótipo

O protótipo foi executado em Windows-11-10.0.26200-SP0, com Python 3.12.14 e Git local. As três execuções hospedadas utilizaram Linux e Python 3.11.16. O workflow usa ubuntu-latest e a série Python 3.11; a comparação entre ambientes é apresentada na seção 4.7. A aplicação é um módulo que calcula o total de um item a partir de preço e quantidade; não representa um serviço web escalável.

A implementação utiliza Decimal para o valor monetário e verifica quantidade inteira positiva e preço não negativo. Seis testes verificam multiplicação, valor zero, arredondamento, rejeição de preço negativo, quantidade zero e quantidade fracionária. A suíte foi executada com unittest, estrutura da biblioteca padrão do Python que permite descoberta e execução de testes.¹³

A aplicação não possui dependências externas. requirements.txt registra essa condição; a preparação executa pip install --no-index -r requirements.txt. Portanto, esta etapa demonstra a presença de uma verificação de dependências vazia, e não a resolução de pacotes de terceiros. Na ampliação do caso, versões e hashes de dependências deverão ser controlados.

O build consiste em verificar sintaticamente o módulo e criar app.zip. Para permitir comparação byte a byte, o pacote contém somente app.py, usa armazenamento sem compressão e uma data interna fixa. O manifesto externo registra commit e SHA-256 do pacote. A igualdade de hashes, portanto, é esperada para o mesmo conteúdo e não demonstra reprodutibilidade de imagens, bibliotecas ou sistemas completos.

O deploy simulado extrai app.zip para staging/<identificador>, com um diretório novo em cada execução. O smoke test importa o módulo desse destino e verifica um cálculo conhecido. Não há publicação em nuvem, servidor persistente, balanceador, banco de dados

ou operação de rollback. A criação exclusiva do diretório evita confundir uma saída antiga com a entrega de uma nova execução.

3.4 Procedimentos de comparação e cenários

No procedimento de referência, as cinco etapas foram invocadas separadamente pelo executor do ensaio, com logs por comando. A denominação "manual de referência" descreve a sequência que um operador precisaria coordenar; não significa que houve observação de uma pessoa trabalhando. Nenhuma taxa de erro humano, tempo de digitação ou esforço cognitivo foi medido.

No procedimento automatizado, um comando inicia `pipeline.py`, que executa as mesmas operações, conserva logs e marca as etapas seguintes como não executadas após a primeira falha. O número de invocações de entrada é uma propriedade do procedimento: cinco no fluxo separado e uma no automatizado. Essa diferença não será convertida em percentual de produtividade.

Quadro 3 - Cenários efetivamente executados

Identificador	Condição	Finalidade
A01, A02 e A03	Mesmo commit e mesma versão válida.	Examinar repetição da sequência e igualdade do pacote.
F01	Troca deliberada da multiplicação por adição.	Verificar detecção do defeito e bloqueio das etapas posteriores.
R01	Restauração do conteúdo original em novo commit.	Observar conclusão do fluxo após correção.
M01	Mesma versão de R01, cinco comandos separados.	Conferir o procedimento de referência e a equivalência do pacote.

Fonte: elaboração própria (2026).

No ensaio local, foram realizadas cinco execuções automatizadas e uma execução por comandos separados. As três repetições iniciais usam o mesmo commit. O cenário de falha altera apenas a operação aritmética; a suíte permanece igual. A recuperação restaura o arquivo original e gera um novo commit, distinguindo a identidade da versão do conteúdo do artefato. Na etapa hospedada, foram executados mais três cenários, em commits próprios, conforme a Tabela 2.

3.5 Critérios de análise e evidências

Quadro 4 - Critérios e evidências observáveis

Critério	Evidência local	Condição para a etapa remota
Padronização	Ordem das cinco etapas e códigos de saída.	Mesma configuração e comandos versionados.
Rastreabilidade	ID de execução, SHA do commit e manifesto.	Relacionar também run ID, evento e URL da execução.
Deteção e bloqueio	Teste falho; etapas seguintes marcadas skipped.	Conferir estado do job e ausência de promoção.
Repetibilidade	Comparação dos hashes dos pacotes.	Repetir no mesmo ambiente e conservar os artefatos.
Intervenção	Invocações previstas no procedimento.	Observar ações humanas com protocolo próprio.
Escalabilidade operacional	Critérios definidos, sem teste de crescimento.	Aumentar mudanças e complexidade; observar filas e consistência.

Fonte: elaboração própria (2026).

Cada execução automatizada gera result.json com identificador, commit, ambiente, versão do Python, códigos de saída, duração técnica das etapas e indicação da existência do destino. As durações foram preservadas nos dados brutos, mas não utilizadas para comparar produtividade, porque não houve pareamento temporal controlado, isolamento do computador ou amostra suficiente. Logs, código e histórico Git são fornecidos em material suplementar.

3.6 Ameaças à validade e procedimentos pendentes

A validade interna é limitada pelo caso único e pelo fato de os procedimentos compartilharem os mesmos comandos. O teste de falha foi escolhido para ser detectável pela suíte, não amostrado de defeitos reais. A validade de construção depende do uso de bloqueio e rastreabilidade como indicadores parciais de confiabilidade. A validade externa é restrita por uma aplicação pequena, sem dependências externas, usuários ou infraestrutura produtiva.

A etapa remota inicial manteve o módulo e a suíte, com instrumentação adicional para registrar run ID, URL e horário UTC. Foram executados os cenários válido, falho e corrigido em Linux e Python 3.11.16. A continuidade deverá repetir cada cenário e ampliar o número de módulos ou ambientes, registrando filas, cancelamentos e falhas. Essa ampliação não integra os resultados já obtidos.

4 RESULTADOS PRELIMINARES

4.1 Evolução em relação ao plano de pesquisa

O avanço desta etapa consiste na transformação do recorte inicial em definições operacionais, critérios de análise, workflow proposto e ensaio local com evidências preservadas. A revisão distingue as práticas contínuas e explicita a diferença entre capacidade de entrega e capacidade de atendimento em produção. A experimentação local verifica a coordenação das etapas; as três execuções hospedadas acrescentam evidência de funcionamento no GitHub Actions, conforme a seção 4.7.

Quadro 5 - Situação dos objetivos específicos

Objetivo	Evolução demonstrada	Situação
Estudar conceitos de CI/CD e DevOps	Distinção conceitual e síntese das condições de adoção.	Desenvolvido nesta etapa.
Identificar recursos do GitHub Actions	Eventos, runners, permissões, retenção e concorrência descritos.	Documentação e execução remota verificadas.
Descrever etapas do pipeline	Cinco etapas implementadas e executadas localmente.	Validações local e remota realizadas.
Analisar impactos operacionais	Repetição, vínculo com commit e bloqueio examinados.	Parcial; sem crescimento de carga.
Investigar limitações	Restrições de testes, ambiente, segurança e plataforma identificadas.	Parcial; sem ensaio de credenciais.
Propor recomendações	Critérios de adoção e protocolo remoto apresentados.	Propostas sujeitas à continuidade.

Fonte: elaboração própria (2026).

4.2 Achados bibliográficos e documentais

A revisão identificou convergência entre a necessidade de feedback verificável e a descrição de fatores técnicos e organizacionais para adotar práticas contínuas. A documentação permite mapear cada requisito do caso a uma configuração ou instrumento: eventos para acionamento; arquivos versionados para definição do fluxo; script para bloqueio; manifesto para vincular artefato e commit; e retenção de evidências para consulta posterior.

Esse mapeamento é um resultado documental e de projeto. Ele não demonstra que toda configuração semelhante produzirá entregas confiáveis. Por exemplo, uma suíte que não verifica uma regra de negócio pode permitir a continuidade de uma alteração incorreta. A

interpretação dos resultados locais exige, portanto, considerar quais propriedades foram efetivamente examinadas.

4.3 Resultados do ensaio local

Tabela 1 - Resultado das execuções exploratórias locais

Execução	Testes aprovados	Build	Deploy simulado	Smoke test
A01	6 de 6	Concluído	Concluído	Aprovado
A02	6 de 6	Concluído	Concluído	Aprovado
A03	6 de 6	Concluído	Concluído	Aprovado
F01	3 de 6	Não executado	Não executado	Não executado
R01	6 de 6	Concluído	Concluído	Aprovado
M01	6 de 6	Concluído	Concluído	Aprovado

Fonte: logs e result.json do ensaio local realizado em 30 set. 2026.

Nas execuções A01, A02 e A03, as cinco etapas concluíram com código de saída zero. Os três pacotes apresentaram o mesmo SHA-256. R01 e M01 também geraram esse pacote, pois o conteúdo de app.py foi restaurado. Isso demonstra igualdade do artefato no escopo e nas condições controladas do ensaio, sem evidenciar determinismo de toda a cadeia de produção.

Em F01, a operação valor * quantidade foi substituída por valor + quantidade. Três testes falharam: multiplicação, arredondamento e preço zero. Para o caso de preço 10 e quantidade 3, a função retornou 13 em vez de 30. O comando de testes retornou código 1; o coordenador marcou build, deploy simulado e smoke como skipped. O diretório staging/F01 não foi criado.

A recuperação R01 restaurou a multiplicação e produziu seis testes aprovados. A geração do pacote, a extração para um novo diretório e o smoke test concluíram. A evidência é compatível com o comportamento de bloquear uma mudança conhecida como incorreta e permitir a continuidade após correção. Não foi testada a reversão de uma entrega já instalada.

4.4 Rastreabilidade e repetibilidade

Quadro 6 - Relação entre versão e execução

Versão	Commit abreviado	Execuções associadas
Inicial válida	38abe9757919	A01, A02 e A03
Defeito deliberado	4a920385aa54	F01
Correção	f404c8cd3504	R01 e M01

Fonte: histórico Git e registros do ensaio local (2026). SHAs completos no Apêndice C.

O commit identifica a versão do repositório; o SHA-256 identifica o conteúdo binário do pacote. São identificadores distintos. A versão inicial e a corrigida possuem commits diferentes, mas geram o mesmo pacote porque o conteúdo empacotado é igual. O manifesto mantém essa distinção e evita utilizar apenas o nome do arquivo como evidência de procedência.

A configuração dos logs foi parte deliberada do protótipo. Um processo manual também pode produzir manifestos e conservar evidências, como ocorreu em M01. Assim, a rastreabilidade observada decorre da instrumentação associada ao versionamento; a automação facilita sua execução coordenada, mas não é a única forma de obtê-la.

4.5 Comparação com o procedimento de referência

Quadro 7 - Comparação descritiva entre procedimentos

Aspecto	Comandos separados (M01)	Script automatizado
Entrada do procedimento	Cinco comandos invocados separadamente.	Um comando aciona a sequência.
Sequenciamento	Coordenação entre invocações pelo executor.	Ordem definida no script versionado.
Tratamento de falha	Decisão de prosseguir depende do procedimento.	Primeiro retorno não zero bloqueia etapas seguintes.
Evidências	Logs preservados pela instrumentação do ensaio.	Logs e resumo gerados pelo script.
Qualidade do pacote válido	Mesmo SHA-256 da versão corrigida.	Mesmo SHA-256 nas execuções válidas.
Erro humano e produtividade	Não medidos.	Não medidos.

Fonte: elaboração própria (2026).

O procedimento separado concluiu com sucesso e gerou o mesmo pacote. A evidência não autoriza afirmar superioridade da automação na qualidade final desse pacote. O contraste demonstrado é procedimental: no script, ordem e bloqueio são executados pelo coordenador; no procedimento separado, a passagem entre comandos precisa ser conduzida pelo executor.

Não foi introduzida uma omissão no procedimento separado nem acompanhado um operador humano. Por essa razão, não se pode estimar quanto a automação reduz erros manuais ou esforço de trabalho. Da mesma forma, não há comparação de tempos totais controlados. A diferença de cinco para uma invocação é apresentada como descrição do protocolo, sem extrapolação para ganho percentual de eficiência.

4.6 Discussão da hipótese e limites dos achados

A hipótese recebe apoio parcial no que diz respeito à explicitação das etapas, à associação de resultados a commits e ao bloqueio após um defeito detectado. As evidências estão circunscritas ao protótipo local. As execuções hospedadas apresentadas na seção 4.7 sustentam a viabilidade do mesmo mecanismo no GitHub Actions. São três cenários exploratórios, insuficientes para generalizações sobre equipes ou sistemas maiores.

O ensaio não demonstra aumento de escalabilidade operacional porque não houve crescimento da frequência ou complexidade das mudanças. A manutenção dos controles em três repetições é uma condição inicial útil, mas insuficiente para essa conclusão. Também não demonstra disponibilidade, segurança global, redução de custos ou qualidade de toda a aplicação. Essas limitações delimitam a interpretação científica e orientam a continuidade.

Os resultados preliminares respondem ao problema ao revelar mecanismos pelos quais a automação pode contribuir: codificar a sequência, executar verificações e conservar evidências associadas à alteração. A extensão desses mecanismos a projetos maiores depende da qualidade dos testes, da preservação dos registros e da validação no ambiente de execução escolhido. A resposta final exige ampliar a comparação remota e examinar o crescimento da frequência e da complexidade previstos no plano.

4.7 Execuções reais e provas públicas no GitHub

O repositório público reúne o código, a configuração do workflow, o histórico de versões e os registros locais e remotos. A publicação permite inspeção por terceiros; a prova da execução hospedada é fornecida pelos links do GitHub Actions, pelos commits associados e pelos artefatos efetivamente produzidos. O endereço do repositório é apresentado também no Apêndice E.

Tabela 2 - Cenários executados no GitHub Actions

Cenário	Run ID	Conclusão	Testes
GH-A01	36754769925	Sucesso	6 de 6
GH-F01	36754802008	Falha deliberada	3 de 6
GH-R01	36754815479	Sucesso	6 de 6

Fonte: execuções e artefatos consultados diretamente no GitHub Actions em 30 set. 2026.

GH-A01 executou a versão válida em main. GH-F01 executou, em branch própria, a substituição deliberada da multiplicação por adição: três dos seis testes falharam, o comando retornou código 1 e as etapas de build, deploy simulado e smoke foram marcadas como skipped. GH-R01 executou uma versão que restaura a multiplicação, com seis testes aprovados e conclusão das etapas. As branches permitem inspecionar a alteração sem introduzir o defeito na versão principal.

As execuções de falha e correção foram acionadas em branches distintas; a ordem de conclusão dependeu da fila da plataforma. Elas demonstram estados de código e controles, sem representar rollback produtivo ou uma sequência temporal de operação em produção. O cenário com falha tem conclusão failure deliberadamente esperada, registrada como controle experimental.

As execuções válidas GH-A01 e GH-R01 produziram o mesmo SHA-256 remoto. Esse valor difere do pacote local: a comparação do conteúdo app.py confirmou equivalência após normalizar CRLF para LF, identificando os finais de linha como a diferença observada. Logo, a repetibilidade foi demonstrada dentro de cada ambiente; identidade binária entre Windows e Linux não foi demonstrada.

Foram conservadas cópias de result.json, dos logs por etapa e do log completo do workflow em evidencias/github/. Elas permitem consultar os resultados mesmo depois da expiração dos artefatos de 14 dias. A seção seguinte mantém a conclusão parcial: três cenários hospedados não medem crescimento de carga ou redução de erros humanos.

5 CONSIDERAÇÕES PARCIAIS E CONTINUIDADE

Nesta etapa, a pesquisa avançou da proposição temática para uma fundamentação conceitual, um protocolo observável e uma prova local de funcionamento. O protótipo demonstrou repetição do pacote, bloqueio de uma falha conhecida e retomada após correção. A referência por comandos separados mostrou que ambos os procedimentos podem produzir o mesmo resultado válido; a automação acrescenta coordenação explícita e execução programada do bloqueio.

Recomenda-se iniciar a adoção por um fluxo pequeno e verificável, associar o artefato ao commit, preservar logs e evitar continuar depois de um teste falho. Na evolução do workflow, a fixação de actions por SHA, a revisão das permissões e uma suíte que represente os riscos reais do projeto devem preceder o uso em produção. Essas recomendações combinam achados locais e critérios documentais, sem substituir validação específica do projeto.

A próxima etapa deverá repetir cada cenário remoto, ampliar a amostra e controlar diferenças de sistema operacional e versão do Python. Os três cenários iniciais já possuem IDs, URLs, commits e cópias dos registros. O estudo poderá incorporar maior número de verificações ou ambientes, acompanhando filas e cancelamentos para investigar a escalabilidade operacional definida no plano.

Quadro 8 - Continuidade prevista no período 2026.2

Período	Atividade prevista	Evidência a produzir
Outubro de 2026	Repetir os cenários já executados no GitHub Actions.	Logs, commits, run IDs, URLs e artefatos.
Outubro de 2026	Ampliar o caso e comparar condições de execução.	Registros de filas, falhas e consistência das etapas.
Outubro e novembro de 2026	Consolidar análise de impactos e limitações.	Matriz comparativa com observações remotas.
Novembro de 2026	Revisar e concluir o relatório.	Resultados consolidados e recomendações finais.

Fonte: adaptação do cronograma do plano de pesquisa para as atividades ainda pendentes (2026).

A conclusão, portanto, permanece parcial: a automação do caso estudado é tecnicamente viável e oferece controles observáveis; o fluxo já funcionou no GitHub Actions, mas seus impactos em escala operacional maior ainda precisam ser demonstrados. A distinção entre resultado observado, inferência analítica e atividade futura é mantida em toda a entrega.

REFERÊNCIAS

1. SOUSA, Gérson Resplandes de Sá. **Plano de pesquisa/investigação: automação de pipelines CI/CD com GitHub Actions e seus impactos na escalabilidade operacional de aplicações**. Balsas: Centro Universitário de Balsas - UNIBALSAS, 2026. Documento acadêmico assinado fornecido para esta pesquisa.
2. FOWLER, Martin. **Continuous Integration**. 18 jan. 2024. Disponível em: <https://martinfowler.com/articles/continuousIntegration.html>. Acesso em: 30 set. 2026.
3. DORA. **Continuous delivery**. [S. l.]: DORA, [s. d.]. Disponível em: <https://dora.dev/capabilities/continuous-delivery/>. Acesso em: 30 set. 2026.
4. UNITED NATIONS. **Goal 9: Industry, Innovation and Infrastructure**. [S. l.]: United Nations, [s. d.]. Disponível em: <https://www.un.org/sustainabledevelopment/infrastructure-industrialization/>. Acesso em: 30 set. 2026.
5. SHAHIN, Mojtaba; BABAR, Muhammad Ali; ZHU, Liming. Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices. **IEEE Access**, v. 5, p. 3909-3943, 2017. DOI: 10.1109/ACCESS.2017.2685629. Disponível em: <https://arxiv.org/abs/1703.07019>. Acesso em: 30 set. 2026.
6. GITHUB. **Understanding GitHub Actions**. GitHub Docs, [s. d.]. Disponível em: <https://docs.github.com/en/actions/get-started/understand-github-actions>. Acesso em: 30 set. 2026.
7. GITHUB. **Workflow syntax for GitHub Actions**. GitHub Docs, [s. d.]. Disponível em: <https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax>. Acesso em: 30 set. 2026.
8. SHAHIN, Mojtaba; ZAHEDI, Mansooreh; BABAR, Muhammad Ali; ZHU, Liming. **An Empirical Study of Architecting for Continuous Delivery and Deployment**. arXiv, 2018. Preprint: arXiv:1808.08796. DOI: 10.48550/arXiv.1808.08796. Disponível em: <https://arxiv.org/abs/1808.08796>. Acesso em: 30 set. 2026.
9. GITHUB. **Control the concurrency of workflows and jobs**. GitHub Docs, [s. d.]. Disponível em: <https://docs.github.com/en/actions/how-tos/write-workflows/choose-when-workflows-run/control-workflow-concurrency>. Acesso em: 30 set. 2026.
10. GITHUB. **Secure use reference**. GitHub Docs, [s. d.]. Disponível em: <https://docs.github.com/en/actions/reference/security/secure-use>. Acesso em: 30 set. 2026.
11. GITHUB. **Store and share data with workflow artifacts**. GitHub Docs, [s. d.]. Disponível em: <https://docs.github.com/en/actions/tutorials/store-and-share-data>. Acesso em: 30 set. 2026.
12. GITHUB. **Actions limits**. GitHub Docs, [s. d.]. Disponível em: <https://docs.github.com/en/actions/reference/limits>. Acesso em: 30 set. 2026.
13. PYTHON SOFTWARE FOUNDATION. **unittest: Unit testing framework**. Python documentation, [s. d.]. Disponível em: <https://docs.python.org/3/library/unittest.html>. Acesso em: 30 set. 2026.

APÊNDICE A - WORKFLOW EXECUTADO NO GITHUB ACTIONS

O arquivo a seguir corresponde a `.github/workflows/ci.yml` no repositório público. Foi executado nos três cenários da seção 4.7. A sequência principal utiliza os mesmos comandos do ensaio local, com instrumentação adicional. As actions estão fixadas por SHA; a execução hospedada utilizou Python 3.11.16 em Linux.

```
name: Pesquisa CI e entrega simulada
on:
  push:
    branches: [main]
    paths:
      - app.py
      - tests/**
      - ops.py
      - pipeline.py
      - requirements.txt
      - .github/workflows/**
  pull_request:
  workflow_dispatch:
permissions:
  contents: read
concurrency:
  group: pesquisa-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
jobs:
  validar:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@11d5960a326750d5838078e36cf38b85af677262 # v4
        with:
          persist-credentials: false
      - uses: actions/setup-python@a26af69be951a213d495a4c3e4e4022e16d87065 # v5
        with:
          python-version: '3.11'
      - name: Testar, empacotar e simular entrega
        run: >-
          python pipeline.py
          run-${{ github.run_id }}-${{ github.run_attempt }}
      - name: Preservar evidencias
        if: always()
        uses: actions/upload-artifact@ea165f8d65b6e75b540449e92b4886f43607fa02 # v4
        with:
          name: evidencias-${{ github.run_id }}-${{ github.run_attempt }}
          path: |
            evidence/
            dist/
          retention-days: 14
```

Fonte: configuração elaborada nesta pesquisa (2026). O pacote suplementar e o repositório contêm os scripts necessários. O ID remoto utiliza `github.run_id` e `github.run_attempt`; o commit é obtido de `GITHUB_SHA`.

APÊNDICE B - ROTEIRO DE REPRODUÇÃO DO ENSAIO

O pacote suplementar contém o código, os logs locais A01-A03, F01, R01 e M01, os registros remotos GH-A01, GH-F01 e GH-R01, os resumos JSON, o arquivo ZIP gerado e um Git bundle com o histórico do experimento. Para reproduzir os resultados, requerem-se Python com pip e Git disponíveis no PATH. O ensaio original usou Python 3.12.14 em Windows; outros ambientes podem produzir diferenças que devem ser registradas.

O bundle conserva os commits e permite clonar as versões sem acesso ao GitHub. Use identificadores novos ao repetir o ensaio: o script recusa reutilizar os diretórios de evidências e destino. Os comandos a seguir devem ser executados no terminal, a partir da pasta que contém `experimento.bundle`:

```
git clone experimento.bundle reproducao
cd reproducao
git checkout 38abe9757919617b399da11f22e88357bdf6b932
python pipeline.py A01_novo
python pipeline.py A02_novo
python pipeline.py A03_novo
git checkout 4a920385aa5477665aaa81b0272c7dec60fb6a40
python pipeline.py F01_novo
# F01_novo deve retornar código 1.
git checkout f404c8cd35042ae132ac8e6115119543462dda2a
python pipeline.py R01_novo
```

Para a referência por comandos separados, utilize a versão corrigida e execute as operações abaixo na ordem apresentada. O executor deve conferir o resultado de cada comando antes de iniciar o seguinte. Isso é parte do protocolo e não foi medido como comportamento humano no ensaio.

```
python -m pip install --no-index -r requirements.txt
python -m unittest discover -s tests -v
python ops.py build
python ops.py deploy M01_novo
python ops.py smoke M01_novo
```

Verifique: seis testes aprovados nas versões válidas; três testes falhos na versão com defeito; ausência de staging/F01_novo; existência dos demais destinos; hashes iguais dos pacotes válidos. O roteiro conserva o contraste entre execução separada e coordenação por script. Não promove qualquer entrega a produção.

APÊNDICE C - REGISTRO SINTÉTICO DAS EVIDÊNCIAS

As evidências abaixo identificam o ensaio de 30 de setembro de 2026. Os SHAs completos evitam ambiguidade na reprodução. A data do ensaio segue o contexto da pesquisa; os timestamps do histórico Git podem refletir o relógio do ambiente de execução.

Versao inicial (A01, A02, A03):
38abe9757919617b399da11f22e88357bdf6b932

Versao com defeito (F01):
4a920385aa5477665aaa81b0272c7dec60fb6a40

Versao corrigida (R01, M01):
f404c8cd35042ae132ac8e6115119543462dda2a

SHA-256 do app.zip nas execucoes validas:
ad950644e42927a8ceeda4087f4f4154
75f5718662713313a430a51d3e4cf39e
(Concatenar as duas linhas para obter os 64 caracteres.)

Trecho do registro de falha, extraído dos logs do cenário F01:

```
test_arredondamento ... FAIL
test_multiplicacao ... FAIL
test_preco_negativo ... ok
test_quantidade_fracionaria ... ok
test_quantidade_zero ... ok
test_zero ... FAIL
Ran 6 tests
FAILED (failures=3)
```

```
testes: exit_code = 1
build: skipped
deploy_simulado: skipped
smoke: skipped
deployment_exists: false
```

Fonte: evidence/F01/testes.log e evidence/F01/result.json. Os caminhos completos do computador foram omitidos deste trecho de apresentação. O pacote suplementar conserva os logs originais.

APÊNDICE D - TRANSPARÊNCIA DA ELABORAÇÃO

Este relatório registra a revisão documental, o ensaio local e três cenários executados no GitHub Actions em 30 de setembro de 2026. Os dados experimentais descrevem execução técnica de comandos e não constituem medições de trabalho humano. Cada origem de evidência possui seus próprios identificadores e registros.

O nome do acadêmico, a matrícula, o orientador, o curso e a unidade curricular foram transcritos do plano fornecido. A instituição e a cidade foram informadas pelo solicitante. Não se presume aprovação do orientador, validação autoral prévia, assinatura ou conclusão da pesquisa. A versão deve ser lida criticamente pelo acadêmico antes de sua submissão.

As fontes externas sustentam conceitos e especificações técnicas; os registros locais e as execuções públicas do GitHub Actions sustentam os resultados do protótipo. As inferências sobre possíveis benefícios são identificadas como parciais. Os livros e o relatório DORA de 2023 citados no plano não foram reproduzidos como referências utilizadas nesta etapa, porque não foram objeto de leitura direta suficiente para fundamentar afirmações específicas.

A apresentação adota papel A4, margens superior e esquerda de 3 cm e inferior e direita de 2 cm, fonte Times New Roman de 12 pontos, corpo com entrelinhas de 1,5 e referências com espaço simples. Utiliza numeração progressiva de seções e sistema numérico de citações, com referências na ordem da primeira ocorrência. A paginação é contada a partir da folha de rosto e exibida a partir da Introdução.

Foram tomadas como base as normas ABNT NBR 14724:2024 (apresentação), NBR 10520:2023 (citações), NBR 6023:2025 (referências), NBR 6024:2012 (seções), NBR 6027:2012 (sumário) e NBR 6028:2021 (resumos), conforme versões identificadas em orientações de bibliotecas universitárias. Trata-se de relatório acadêmico parcial, sem folha de aprovação ou ficha catalográfica, pois não há defesa ou aprovação registrada. Eventuais exigências institucionais complementares precisam ser consideradas na submissão.

Consulta de normalização: Sistema Integrado de Bibliotecas da Universidade Federal do Espírito Santo.
Disponível em: <https://biblioteca.ufes.br/normalizacao>. Acesso em: 30 set. 2026.

APÊNDICE E - LINKS PARA CONFERÊNCIA DAS PROVAS

Os links abaixo são clicáveis. Para conferir o experimento, consulte o código, o índice de provas, o commit de preservação das evidências e as três execuções do GitHub Actions. A página do repositório é a entrada principal; o commit abaixo identifica uma versão fixa dos registros.

Repositório público

<https://github.com/GersonResplandes/pesquisa-cicd-github-actions>

Índice de provas

<https://github.com/GersonResplandes/pesquisa-cicd-github-actions/blob/4786f45ee87c2af75af69bb47cc2465590c7fc54/evidencias/PROVAS.md>

Commit que preserva os registros

<https://github.com/GersonResplandes/pesquisa-cicd-github-actions/commit/4786f45ee87c2af75af69bb47cc2465590c7fc54>

GH-A01 - execução aprovada

<https://github.com/GersonResplandes/pesquisa-cicd-github-actions/actions/runs/36754769925>

GH-F01 - falha deliberada

<https://github.com/GersonResplandes/pesquisa-cicd-github-actions/actions/runs/36754802008>

GH-R01 - execução aprovada

<https://github.com/GersonResplandes/pesquisa-cicd-github-actions/actions/runs/36754815479>

Roteiro de conferência: confirmar o commit e a conclusão de cada execução; abrir o job validar; examinar testes.log e result.json; verificar o bloqueio em GH-F01 e a conclusão nas duas versões válidas. Os logs copiados têm origem declarada no índice de provas. Os SHAs locais permanecem no Apêndice C e não devem ser confundidos com os commits dos três cenários hospedados.